

4-3 預測房價：迴歸範例

Google colab檔案：[預測房價-實作.ipynb](#)

前言：

迴歸(regression)問題，模型需要預測一個連續而不是離散標籤

例如：氣象資料預測明天溫度等

*迴歸(regression)和邏輯斯迴歸(Logistic regression)不一樣，邏輯斯迴歸(Logistic regression)是一種分類演算法

4-3-1: 載入波士頓住房價格資料集

將標籤分別存成train_targets和test_targets(訓練標籤，測試標籤)

```
from tensorflow.keras.datasets import boston_housing
(train_data, train_targets), (test_data, test_targets) = boston_housing.load_data()
# 將標籤分別存成train_targets和test_targets(訓練標籤，測試標籤)
# tensorflow.keras.datasets 用於載入預設資料集，如本例中的波士頓住房價格資料集。

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/boston_housing.npz
57026/57026 ----- 0s 1us/step
```

查看訓練資料的shape

shape=資料結構[特別是陣列或張量]的維度大小，它描述了資料有多少「行」和多少「列」

```
▶ train_data.shape
... (404, 13)
```

查看測驗資料的shape

shape=資料結構[特別是陣列或張量]的維度大小，它描述了資料有多少「行」和多少「列」

```
▶ test_data.shape
... (102, 13)
```

查看訓練資料的目標值，就是實際成交房價的中位數

```
▶ train_targets #訓練資料的目標值，就是實際成交房價的中位數
... array([15.2, 42.3, 50. , 21.1, 17.7, 18.5, 11.3, 15.6, 15.6, 14.4, 12.1,
          17.9, 23.1, 19.9, 15.7,  8.8, 50. , 22.5, 24.1, 27.5, 10.9, 30.8,
          32.9, 24. , 18.5, 13.3, 22.9, 34.7, 16.6, 17.5, 22.3, 16.1, 14.9,
          23.1, 34.9, 25. , 13.9, 13.1, 20.4, 20. , 15.2, 24.7, 22.2, 16.7,
          12.7, 15.6, 18.4, 21. , 30.1, 15.1, 18.7,  9.6, 31.5, 24.8, 19.1,
          22. , 14.5, 11. , 32. , 29.4, 20.3, 24.4, 14.6, 19.5, 14.1, 14.3,
          15.6, 10.5,  6.3, 19.3, 19.3, 13.4, 36.4, 17.8, 13.5, 16.5,  8.3,
          14.3, 16. , 13.4, 28.6, 43.5, 20.2, 22. , 23. , 20.7, 12.5, 48.5,
          14.6, 13.4, 23.7, 50. , 21.7, 39.8, 38.7, 22.2, 34.9, 22.5, 31.1,
          28.7, 46. , 41.7, 21. , 26.6, 15. , 24.4, 13.3, 21.2, 11.7, 21.7,
          19.4, 50. , 22.8, 19.7, 24.7, 36.2, 14.2, 18.9, 18.3, 20.6, 24.6,
```

4-3-2: 正規化資料

將大小範圍差異很大的數值(某特徵的值是三位數, 另一個卻是個位數)直接輸入神經網路, 通常會增加學習困難度, 降低訓練成效, 因此必須對資料進行正規化, 也就是對輸入資料中的每個特徵值(輸入資料矩陣中的一欄)減去該特徵的平均值並除以標準差, 這樣一來特徵數值會以0為中心

*對測試資料正規化, 必須使用訓練資料集來計算, 否則會有資料洩漏的問題

```
mean = train_data.mean(axis=0)#沿著第0軸(樣本軸)計算平均值
train_data -= mean
std = train_data.std(axis=0)#沿著第0軸(樣本軸)計算標準差
train_data /= std #需使用訓練資料集否則會有資料漏洩問題

#正規化處理
test_data -= mean
test_data /= std
```

4-3-3: 建立模型(模型定義)

由於此範例可用的樣本數很少, 我們使用小型的神經網路, 因訓練資料越少, 過度配適情況會越嚴重(因為很容易就會被記憶起來), 而使用小型的神經網路是緩解過度配適的方法之一。過度配適是指模型在訓練資料上表現得非常好, 但在面對從未見過的新資料(例如測試資料或實際應用中的資料)時, 表現卻非常差的現象

```
from keras import models
from keras import layers
import keras

def build_model():
    model = keras.Sequential([
        layers.Dense(64, activation="relu"),
        layers.Dense(64, activation="relu"),
        layers.Dense(1)
    ])
    model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
    #使用mse均方差(預測值和目標值之間差異的平方值)
    #mae平均絕對誤差(預測值和目標值之間差異的絕對值)
    return model
```

4-3-3: K折交叉驗證

K折驗證說明

a. 說明: 首先將數據集切割成k組, 然後輪流在k組中挑選一組作為測試集, 其它都為訓練集, 然後執行測試, 進行了k次後, 將每次的測試結果平均起來, 就為在執行k折交叉驗證法 (k-fold Cross Validation)下模型的性能指標

b. k取值: 通常都取10, 資料量大時, 建議設小, 反之設大, 這樣在資料數量小的時候能多分一點數據給訓練集來訓練模型

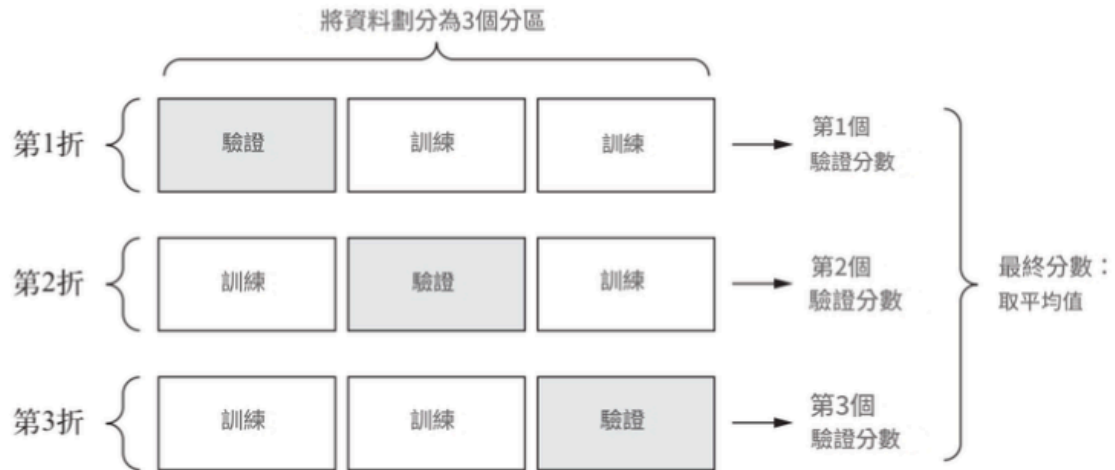


圖3-11 3折交叉驗證

```
import numpy as np
k = 4 #進行4折交叉驗證
num_val_samples = len(train_data) // k #計算驗證集的樣本數
num_epochs = 100
all_scores = []
for i in range(k):
    print(f"Processing fold #{i}")
    #準備驗證資料:資料來自第i個區塊
    val_data = train_data[i * num_val_samples: (i + 1) * num_val_samples]
    val_targets = train_targets[i * num_val_samples: (i + 1) * num_val_samples]

    #準備訓練資料:資料來自第i個以外的所有區塊
    partial_train_data = np.concatenate(
        [train_data[i * num_val_samples:],
         train_data[(i + 1) * num_val_samples:]],
        axis=0)
    partial_train_targets = np.concatenate(
        [train_targets[i * num_val_samples:],
         train_targets[(i + 1) * num_val_samples:]],
        axis=0)

    #重新建立Keras模型(已編譯過)
    model = build_model()
    #訓練該模型(在silent靜音模式下, verbose = 0)
    model.fit(partial_train_data, partial_train_targets,
              epochs=num_epochs, batch_size=16, verbose=0)

    #以驗證資料來評估模型
    val_mse, val_mae = model.evaluate(val_data, val_targets, verbose=0)
    all_scores.append(val_mae)
```

```
*** Processing fold # 0
Processing fold # 1
Processing fold # 2
Processing fold # 3
```

num_epochs = 100(100個週期)產生的結果

```
all_scores
... [1.91187584400177, 2.519096851348877, 2.586074113845825, 2.3349690437316895]

np.mean(all_scores)

np.float64(2.3380039632320404)
```

代表著K折驗證的平均分是2.34，也就是價格誤差為\$2,338美元之間，反映出模型成效不是很好

因此我們嘗試以更長的時間(500週期)來訓練神經網路

(在Keras的model.fit()函數中，verbose=0的意思是將訓練過程設定為「靜音模式」。這表示在模型訓練的每個週期(epoch)中，它不會輸出任何進度條、損失值(loss)或評估指標(metrics)等資訊。這樣做的好處是：

減少輸出訊息：當你執行多次訓練或訓練時間較短時，可以避免螢幕被大量的訓練日誌淹沒。
提高執行速度：在某些情況下，輸出資訊的處理也可能佔用少量資源，設定為靜音模式可以輕微提升訓練速度。當verbose設定為其他值時：

verbose=1 (預設值)：顯示進度條和每週期的訓練結果。

verbose=2：只顯示每週期的訓練結果，不顯示進度條。)

```
num_epochs = 500
all_mae_histories = []
for i in range(k):
    print(f"Processing fold #{i}")
    val_data = train_data[i * num_val_samples: (i + 1) * num_val_samples]
    val_targets = train_targets[i * num_val_samples: (i + 1) * num_val_samples]
    partial_train_data = np.concatenate(
        [train_data[i * num_val_samples:
                    (i + 1) * num_val_samples:]],
        axis=0)
    partial_train_targets = np.concatenate(
        [train_targets[i * num_val_samples:
                    (i + 1) * num_val_samples:]],
        axis=0)
    model = build_model()
    history = model.fit(partial_train_data, partial_train_targets,
                        validation_data=(val_data, val_targets),
                        epochs=num_epochs, batch_size=16, verbose=0)
    #訓練該模型(在silent靜音模式下，verbose=0)
    mae_history = history.history["val_mae"]
    all_mae_histories.append(mae_history)

... Processing fold #0
Processing fold #1
Processing fold #2
Processing fold #3
```

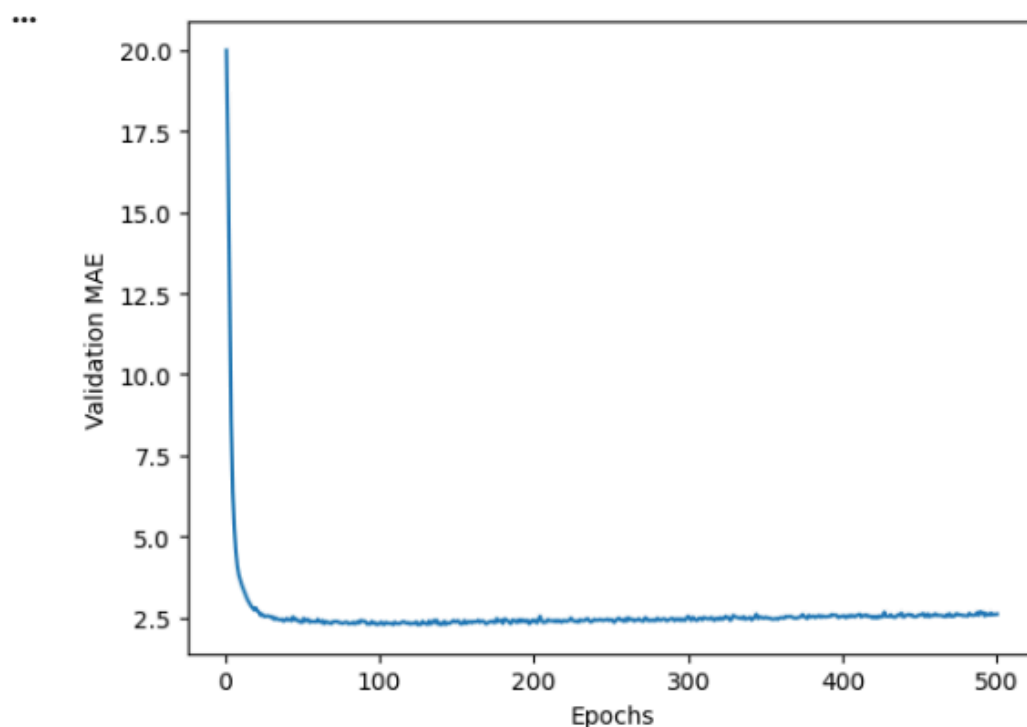
建立連續K折驗證分數的歷史

```
average_mae_history = [
    np.mean([x[i] for x in all_mae_histories]) for i in range(num_epochs)]
```

讓我們把MAE(平均絕對誤差)分數給畫出來

```
import matplotlib.pyplot as plt

plt.plot(range(1, len(average_mae_history) + 1), average_mae_history)
plt.xlabel("Epochs")
plt.ylabel("Validation MAE")
plt.show()
```



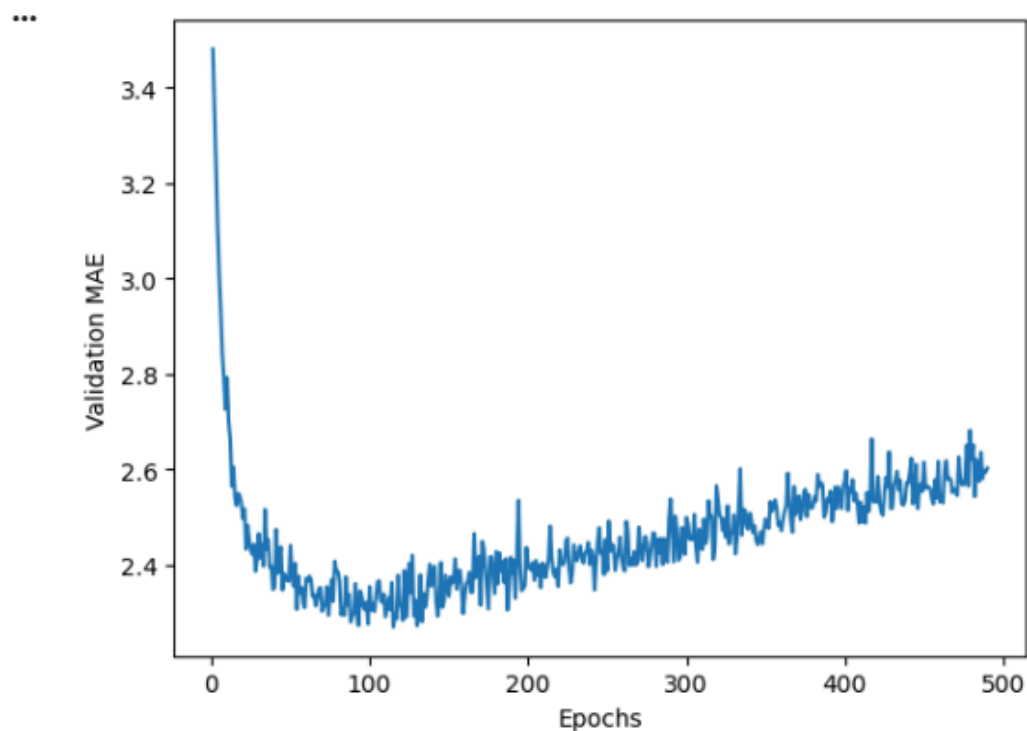
由於單位刻度縮放問題的相對較高的變異數, 因此我們排除前10個資料點, 因為這些資料點與曲線其他部分差異過大

1. 單位刻度縮放問題 (Unit Scale Problem): 想像一下, 你的資料集中有一個特徵代表「房屋面積」(例如: 1000平方英尺), 另一個特徵代表「房間數量」(例如: 3個)。這些特徵的數值範圍相差很大。一個特徵的值可能是幾千, 而另一個特徵的值只是個位數。
2. 相對較高的變異數 (Relatively High Variance): 在這種情況下, 「變異數」可能指兩個層面:
 - 特徵本身的變異數: 如果一個特徵的數值範圍很大(例如: 0到10000), 它的統計變異數自然會比一個數值範圍很小(例如: 0到10)的特徵大得多。
 - 模型訓練過程中的變異數/不穩定性: 當特徵的刻度差異很大時, 機器學習模型(特別是那些基於梯度下降的優化算法)會遇到困難。因為梯度下降會更傾向於對數值範圍較大的特徵做出更大的調整, 這可能導致:
 - 收斂速度變慢: 模型需要更長的時間才能找到最佳解。
 - 訓練過程不穩定: 模型可能會在最佳解附近震盪, 難以精確收斂。
 - 某些特徵被「過度強調»: 模型可能會錯誤地認為數值範圍大的特徵比數值範圍小的特徵更重要, 即使實際上並非如此。

```

truncated_mae_history = average_mae_history[10:]
plt.plot(range(1, len(truncated_mae_history) + 1), truncated_mae_history)
plt.xlabel("Epochs")
plt.ylabel("Validation MAE")
plt.show()

```



可以看出MAE在120-140個週期後停止改善，然後開始往上升，換句話說過了120-140便開始過度配適了，除了週期外我們還可以調整隱藏層的大小。

```

model = build_model() #建立一個用最佳參數編譯過的新模型
model.fit(train_data, train_targets, #以所有資料進行訓練
          epochs=130, batch_size=16, verbose=0)
test_mse_score, test_mae_score = model.evaluate(test_data, test_targets)

```

4/4 ----- 0s 7ms/step - loss: 13.5323 - mae: 2.6302

最終結果

```
test_mae_score
```

```
2.7545955181121826
```

4-3-5: 新資料上進行預測

```
predictions = model.predict(test_data)
predictions[0]
```

```
4/4 ----- 0s 12ms/step
array([9.376246], dtype=float32)
```

模型對於測試資料集中首間房屋的預測價格為9,000美元

4-3-6: 小結

1. 迴歸問題的損失函數與分類問題不同，經常使用均方標準差
2. 常見的迴歸評估指標是平均絕對誤差(MAE)
3. 當輸入資料的特徵具有不同的度量刻度與數值範圍，應預先將每個特徵進行單獨的數值範圍轉換，如正規化處理
4. 可用資料很少時，K折驗證是評估模型的好方法
5. 當可用訓練資料很少時，最好使用隱藏層較少的小型神經模型，以免造成過度配適的問題